



Python Programming

Complete Beginner Notes

Basics · OOPs · Data Structures · File Handling & More

1. Introduction to Python

Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability. Created by Guido van Rossum in 1991, it follows a 'batteries included' philosophy.

- ✓ Easy to learn and read
- ✓ Interpreted (line by line)
- ✓ Dynamically typed
- ✓ Cross-platform (Windows/Linux/Mac)
- ✓ Open source & free
- ✓ Object-Oriented + Functional
- ✓ Huge standard library

1.1 Running Python

You can run Python in three ways:

- ▶ Interactive Mode: Type python in terminal → get >>> prompt
- ▶ Script Mode: Write code in .py file → run with python filename.py
- ▶ IDE Mode: Use tools like VS Code, PyCharm, Jupyter Notebook

2. Variables & Data Types

A variable is a name that stores a value. Python is dynamically typed — you don't need to declare types.

```
name = 'Alice'      # str
age  = 25           # int
gpa  = 9.2          # float
is_student = True   # bool
nothing = None      # NoneType
```

2.1 Built-in Data Types

Data Type	Description & Example
-----------	-----------------------

int	Whole numbers → 10, -5, 1000
float	Decimal numbers → 3.14, -0.5
str	Text in quotes → 'hello', "world"
bool	True or False only
list	Ordered, mutable collection → [1, 2, 3]
tuple	Ordered, immutable collection → (1, 2, 3)
set	Unordered, unique items → {1, 2, 3}
dict	Key-value pairs → {'name': 'Alice'}
None	Represents absence of value

2.2 Type Conversion

```
int('42')      # → 42
float('3.14')  # → 3.14
str(100)       # → '100'
bool(0)        # → False
list((1,2,3))  # → [1, 2, 3]
```

3. Operators

3.1 Arithmetic Operators

Operator	Description
+	Addition → 5 + 3 = 8
-	Subtraction → 5 - 3 = 2
*	Multiplication → 5 * 3 = 15
/	Division (float) → 7 / 2 = 3.5
//	Floor Division → 7 // 2 = 3
%	Modulus (remainder) → 7 % 2 = 1
**	Exponent → 2 ** 3 = 8

3.2 Comparison & Logical Operators

```
# Comparison → returns True/False
10 == 10    # Equal
10 != 5     # Not equal
10 > 5      # Greater than
10 < 20     # Less than

# Logical
```

```
True and False    # → False
True or  False    # → True
not True          # → False
```

4. Control Flow

4.1 if / elif / else

```
marks = 75
if marks >= 90:
    print('Grade: A')
elif marks >= 75:
    print('Grade: B')
else:
    print('Grade: C')
```

4.2 for Loop

```
# Iterating over a list
fruits = ['apple', 'banana', 'mango']
for fruit in fruits:
    print(fruit)

# Using range()
for i in range(1, 6):    # 1 to 5
    print(i)
```

4.3 while Loop

```
count = 0
while count < 5:
    print('Count:', count)
    count += 1
```

4.4 break, continue, pass

Keyword	Purpose
break	Exit the loop immediately
continue	Skip current iteration, go to next
pass	Do nothing (placeholder)

5. Functions

A function is a reusable block of code. Defined with the `def` keyword.

```
def greet(name, greeting='Hello'):    # default argument
    return f'{greeting}, {name}!'

print(greet('Alice'))                # Hello, Alice!
print(greet('Bob', 'Hi'))            # Hi, Bob!
```

5.1 *args and **kwargs

```
def total(*args):                    # variable positional args
    return sum(args)

def display(**kwargs):              # variable keyword args
    for k, v in kwargs.items():
        print(k, '=', v)

total(1, 2, 3, 4)                   # → 10
display(name='Alice', age=25)
```

5.2 Lambda (Anonymous) Functions

```
square = lambda x: x ** 2
print(square(5))                    # → 25

add = lambda a, b: a + b
print(add(3, 4))                    # → 7
```

6. Strings

6.1 String Operations

```
s = 'Hello, Python!'
len(s)                             # → 15
s.upper()                          # → 'HELLO, PYTHON!'
s.lower()                          # → 'hello, python!'
s.replace('Python', 'World')       # → 'Hello, World!'
s.split(',')                       # → ['Hello', 'Python!']
s.strip()                          # removes leading/trailing spaces
'py' in s                          # → False (case sensitive)
```

6.2 String Formatting

```
name = 'Alice'
age = 25
```

```
# f-string (recommended)
print(f'Name: {name}, Age: {age}')

# .format()
print('Name: {}, Age: {}'.format(name, age))

# % operator (old style)
print('Name: %s, Age: %d' % (name, age))
```

6.3 String Slicing

```
s = 'Python'
s[0]      # → 'P'      (first char)
s[-1]     # → 'n'      (last char)
s[0:3]    # → 'Pyt'    (index 0,1,2)
s[:2]     # → 'Pt'     (every 2nd char)
s[::-1]   # → 'nohtyP' (reversed)
```

7. Data Structures

7.1 List

```
nums = [10, 20, 30, 40]
nums.append(50)      # add at end
nums.insert(1, 15)   # insert at index 1
nums.remove(30)      # remove by value
nums.pop()           # remove last element
nums.sort()          # sort ascending
nums.reverse()       # reverse list
len(nums)            # number of elements
```

7.2 Tuple

```
t = (1, 2, 3, 4)
t[0]      # → 1      (access)
len(t)    # → 4
t.count(2) # → 1      (occurrences)
t.index(3) # → 2      (position)
# Tuples are immutable – cannot change after creation
```

7.3 Dictionary

```
student = {'name': 'Alice', 'age': 25, 'gpa': 9.2}
student['name']      # → 'Alice'
student['city'] = 'Lucknow' # add new key
```

```
student.get('phone', 'N/A')    # safe access
student.keys()                 # → dict_keys([...])
student.values()               # → dict_values([...])
student.items()                # → key-value pairs
del student['gpa']              # delete a key
```

7.4 Set

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
s1.add(7)                      # add element
s1.remove(1)                   # remove element
s1 | s2                        # union
s1 & s2                        # intersection
s1 - s2                        # difference
```

7.5 List Comprehension

```
# Squares of 0-9
squares = [x**2 for x in range(10)]

# Even numbers only
evens = [x for x in range(20) if x % 2 == 0]

# Dict comprehension
cube_map = {x: x**3 for x in range(5)}
```

8. Object-Oriented Programming (OOP)

OOP is a programming paradigm that organizes code into objects. Python fully supports OOP with four main pillars.

The 4 Pillars of OOP

1. Encapsulation — Bundling data + methods; hiding internal state
2. Inheritance — Child class inherits from parent class
3. Polymorphism — Same method name, different behavior
4. Abstraction — Hiding complexity, showing only essentials

8.1 Class & Object

```
class Student:
    # Class variable (shared by all objects)
    school = 'HCLTech Academy'
```

```

# Constructor / Initializer
def __init__(self, name, age):
    self.name = name    # instance variable
    self.age = age

# Instance method
def introduce(self):
    return f'Hi, I am {self.name}, age {self.age}'

# Creating objects
s1 = Student('Alice', 22)
s2 = Student('Bob', 25)
print(s1.introduce())    # Hi, I am Alice, age 22

```

8.2 Encapsulation

Encapsulation hides data using private (__) and protected (_) attributes.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):           # getter
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())    # 1500
# print(acc.__balance)    # Error! Private

```

8.3 Inheritance

A child class inherits attributes and methods from a parent class.

```

class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return 'Some sound'

class Dog(Animal):                # Dog inherits Animal
    def speak(self):             # method overriding
        return 'Woof!'

```

```

class Cat(Animal):
    def speak(self):
        return 'Meow!'

d = Dog('Bruno')
c = Cat('Kitty')
print(d.speak())    # Woof!
print(c.speak())    # Meow!

```

8.4 Multiple Inheritance

```

class Father:
    def skills(self): return 'Cooking'

class Mother:
    def skills(self): return 'Painting'

class Child(Father, Mother):
    pass

c = Child()
print(c.skills())    # Cooking    (MRO: left to right)

```

8.5 Polymorphism

Same interface — different implementations. Polymorphism allows functions to work with any object that has the required method.

```

class Shape:
    def area(self): pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h): self.w=w; self.h=h
    def area(self): return self.w * self.h

shapes = [Circle(5), Rectangle(4, 6)]
for s in shapes:
    print(s.area())    # 78.5, 24

```

8.6 Abstraction (ABC Module)

```

from abc import ABC, abstractmethod

```



```

class Vehicle(ABC):      # abstract class
    @abstractmethod
    def fuel_type(self): # must override in child
        pass

class Car(Vehicle):
    def fuel_type(self):
        return 'Petrol'

# Vehicle()    # Error! Can't instantiate abstract class
c = Car()
print(c.fuel_type())    # Petrol

```

8.7 Special / Dunder Methods

Method	Purpose
<code>__init__</code>	Constructor — called when object is created
<code>__str__</code>	String representation → used by <code>print()</code>
<code>__repr__</code>	Official string representation for debugging
<code>__len__</code>	Called by <code>len()</code>
<code>__add__</code>	Operator overloading for <code>+</code>
<code>__eq__</code>	Operator overloading for <code>==</code>
<code>__del__</code>	Destructor — called when object is deleted

```

class Point:
    def __init__(self, x, y):
        self.x = x; self.y = y
    def __str__(self):
        return f'Point({self.x}, {self.y})'
    def __add__(self, other):
        return Point(self.x+other.x, self.y+other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)    # Point(4, 6)

```

9. Exception Handling

Exception handling prevents program crashes due to runtime errors.

```

try:
    num = int(input('Enter a number: '))

```

```

    result = 100 / num
    print('Result:', result)
except ValueError:
    print('Invalid input! Enter a number.')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print('Unexpected error:', e)
else:
    print('No errors occurred!')    # runs if no exception
finally:
    print('Always runs!')          # cleanup code

```

9.1 Custom Exception

```

class AgeError(Exception):
    pass

def check_age(age):
    if age < 0:
        raise AgeError('Age cannot be negative!')
    return age

try:
    check_age(-5)
except AgeError as e:
    print(e)    # Age cannot be negative!

```

10. File Handling

```

# Writing to a file
with open('notes.txt', 'w') as f:
    f.write('Hello, Python!\n')
    f.write('File handling is easy.')

# Reading from a file
with open('notes.txt', 'r') as f:
    content = f.read()    # entire file
    # OR
    lines = f.readlines() # list of lines

# Appending
with open('notes.txt', 'a') as f:
    f.write('\nNew line added.')

```

Mode	Description
'r'	Read (default) — file must exist
'w'	Write — creates/overwrites file
'a'	Append — adds to end of file
'rb'	Read in binary mode
'x'	Create — fails if file exists

11. Modules & Packages

```
import math
import random
from os import path
import datetime

print(math.sqrt(16))      # → 4.0
print(math.pi)           # → 3.14159...
print(random.randint(1,10)) # → random int
print(datetime.date.today()) # → current date
```

11.1 Creating Your Own Module

```
# File: mymath.py
def add(a, b): return a + b
def multiply(a, b): return a * b
PI = 3.14159

# File: main.py
import mymath
print(mymath.add(3, 4))      # → 7
print(mymath.PI)            # → 3.14159
```

12. Useful Built-in Functions

Function	Description
print()	Output to console
input()	Take user input (returns string)
len()	Length of string/list/etc.
range()	Generate sequence of numbers
type()	Return type of an object

<code>int/float/str()</code>	Type conversion
<code>sorted()</code>	Return sorted list
<code>enumerate()</code>	Loop with index → (index, value)
<code>zip()</code>	Combine two iterables
<code>map()</code>	Apply function to all items
<code>filter()</code>	Filter items by condition
<code>sum()</code>	Sum of all elements
<code>min()/max()</code>	Smallest/Largest element
<code>abs()</code>	Absolute value
<code>round()</code>	Round a float
<code>id()</code>	Memory address of object
<code>dir()</code>	List attributes of an object
<code>help()</code>	Display documentation

13. Iterators & Generators

13.1 Iterators

```
my_list = [1, 2, 3]
it = iter(my_list)      # create iterator
print(next(it))         # → 1
print(next(it))         # → 2
```

13.2 Generators

Generators use `yield` instead of `return` — they produce values lazily (one at a time), saving memory.

```
def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1

for val in count_up(5):
    print(val)    # 0 1 2 3 4

# Generator expression (like list comprehension)
gen = (x**2 for x in range(10))
```

14. Decorators

A decorator is a function that wraps another function to add behavior.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}()')
        result = func(*args, **kwargs)
        print(f'Done!')
        return result
    return wrapper

@logger          # same as: greet = logger(greet)
def greet(name):
    print(f'Hello, {name}!')

greet('Alice')
# Calling greet()
# Hello, Alice!
# Done!
```

15. Quick Reference Cheat Sheet

DATA TYPES: int float str bool list tuple set dict None

OPERATORS: + - * / // % ** == != > < and or not in is

KEYWORDS: if elif else for while break continue pass

def return lambda class try except finally raise

import from as with yield

OOP PILLARS: Encapsulation Inheritance Polymorphism Abstraction

ACCESS: public _protected __private

FILE MODES: r w a rb wb x

— End of Python Notes —